WATCHLIST MANAGER

LINUX OS LAB PROJECT REPORT

A menu-driven Bash application for managing movies and web series

Student Name	Zarmin Shaikh
Roll Number	F07
Division	G
Enrollment Number	24004501210415
University	LJ University
Subject	OS Linux Lab
Semester	5
Project Name	Watchlist Manager

Semester 5

1. Introduction and Objectives

Project purpose

Watchlist Manager is a menu-driven Linux application written in Bash Shell Scripting. It helps a user maintain a CSV list of movies and web series, mark titles as watched, save ratings, find pending titles by genre, calculate average ratings, create backups, and review activity logs.

Objectives

The objectives are to practise Bash input handling, validate user entries, search and update text data, perform simple calculations, archive files, record actions, and run a scheduled recommendation task.

How the menu works

The interactive menu accepts a menu number and calls the related operation. Separate command-line options are documented for help, backup, and the weekend recommendation task; these are terminal commands, not menu selections.

2. Project Folder Structure

```
watchlist/
■■■ main.sh
■■■ README.md
■■■ lib/functions.sh
■■■ data/watchlist.csv
■■■ backups/
■■■ logs/activity.log
■■■ reports/ (if present)
■■■ weekend_picks.txt
```

main.sh — Main entry point. Displays the menu and handles documented command-line options.

README.md — Project overview and run instructions.

lib/functions.sh — Reusable functions for watchlist operations.

data/watchlist.csv — CSV data file containing movie/series records and their status/rating fields.

backups/ — Destination for generated compressed backup archives.

logs/activity.log — Append-only activity history created/updated by logging operations.

weekend_picks.txt — Output file for the weekend recommendation task.

reports/ (if present) — A report folder appears in the supplied reference structure; verify whether it exists in your working copy.

Generated files, including archives, logs, and weekend recommendations, may appear or change after running the application.

3. Feature Logic and Output

Menu numbers refer to options selected inside the interactive menu. Commands in the run section are typed directly into the terminal.

1. Add a Title

Purpose: Add a movie or series to the CSV watchlist.

Working / Logic:

1. `read` asks for the title, type, and genre and stores each answer in a shell variable.

2. `-z` checks whether an input string is empty. If a required value is empty, the script rejects the entry instead of saving an incomplete record.

3. `cut -d',' -f1` splits each CSV line at commas and extracts field 1 (the title) for duplicate checking.

4. `grep -iFxq` searches the extracted titles without case sensitivity (`-i`), requires a whole-line exact match (`-x`), and stays quiet (`-q`). A match means the title already exists.

5. If the inputs are acceptable and the title is not a duplicate, `echo ... >> "$DATA"` appends a new CSV record. The record begins with `pending` status and an empty rating.

6. `log()` records that the add operation happened.

Commands used: `read`, `-z`, `cut -d',' -f1`, `grep -iFxq`, `echo`, `>>`, `log()`.

Output explanation: “Saved!” is shown after the script reaches its successful-save path. Empty required fields or a duplicate title are rejected, so no new record should be appended in those cases.

Screenshot placement: Run `cd ~/linux_project/watchlist`, then `./main.sh`; choose menu option 1 and enter a new title, type, and genre. Capture the success message. Use a test title that is not already in the CSV.

[Insert your own terminal screenshot here]

2. Mark as Watched and Rate

Purpose: Change an existing title's status to watched and store its rating.

Working / Logic:

1. `read` collects the title and rating. `grep` checks whether the requested title exists in the data file.

2. The rating is checked against the permitted range of 1 through 10. An invalid rating is rejected; it should not be saved as a valid rating.

3. When the title exists and the rating is valid, `sed -i` edits the file in place, changing the selected record's status to watched and saving the entered rating.

4. `grep` is then used to display the updated matching record, so the user can see the changed data.

5. `log()` records the update.

Commands used: `read`, `grep`, rating condition, `sed -i`, `log()`.

Output explanation: A found title with a valid rating should be shown with watched status and the new rating. If the title is not found, the script reports that it cannot update that title. If the rating is outside 1–10 or otherwise fails the script's validation, the update is rejected. Exact wording depends on the source script.

Screenshot placement: Run the menu and select option 2. Enter an existing title and a rating from 1 to 10. Capture the displayed updated record. Do not use a made-up output in place of your terminal screenshot.

[Insert your own terminal screenshot here]

3. Recommend Unwatched Titles by Genre

Purpose: List pending titles that match a genre entered by the user.

Working / Logic:

1. `read` accepts the genre name.

2. `grep -i` searches without case sensitivity. The search pattern is intended to match the requested genre together with `pending` status, so watched records are not recommended.

3. Matching records are printed to the terminal. If the search returns no matching records, the program prints “Nothing found.”

4. `log()` records the search activity.

Commands used: `read`, `grep -i`, `log()`.

Output explanation: Only records that match the genre and pending status should appear. “Nothing found.” means the search did not find any records matching those conditions.

Illustrative example: Illustrative example only: entering `Mystery` may list pending Mystery titles if such records exist. This is not claimed to be the actual data or output in the uploaded report.

Screenshot placement: Select menu option 3 and enter a genre that is present among pending records in your own CSV. Capture the matching lines. If there are no matches, capture the “Nothing found.” message instead.

[Insert your own terminal screenshot here]

4. Average Rating per Genre

Purpose: Calculate the average rating for each genre using rated, watched titles.

Working / Logic:

1. `awk -F','` reads each CSV line and treats commas as field separators.

2. The calculation includes records whose status is watched and whose rating field is not empty.

3. `sum[genre]` adds each qualifying rating to that genre's total. `count[genre]` increases once for each qualifying rated title.

4. For each genre, the average is `sum[genre] / count[genre]`.

5. `printf "%.2f"` formats the calculated average to two digits after the decimal point.

Commands used: `awk -F','`, `sum[genre]`, `count[genre]`, `printf "%.2f"`.

Output explanation: Each displayed average summarizes only the qualifying watched records in that genre. If a genre has no watched titles with ratings, it should not contribute a calculated average.

Illustrative example: Illustrative calculation: Comedy ratings 9 and 7 give a total of 16; 16 divided by 2 titles = 8.00. This is a teaching example, not a claim about the actual CSV.

Screenshot placement: Select menu option 4. Capture the average-rating results printed from your current data file.

[Insert your own terminal screenshot here]

5. Backup Watchlist with Date Stamp

Purpose: Create a timestamped compressed backup of the data directory.

Working / Logic:

1. `date +%Y%m%d_%H%M%S` produces a timestamp in year-month-day and time form, such as `20261029_190000` (format example only).
2. `tar -czf` creates a gzip-compressed archive containing the `data/` directory.
3. The archive is placed inside `backups/` and the terminal displays the generated backup path.
4. `log()` records the backup operation.

Commands used: `date +%Y%m%d_%H%M%S`, `tar -czf`, `log()`.

Output explanation: The displayed path identifies where the archive was written. The actual filename depends on the time the command runs; any sample filename in this report is illustrative only.

Illustrative example: Example filename pattern only: `backups/watchlist_20261029_190000.tar.gz`.

Screenshot placement: Select menu option 5 and capture the backup path printed by your terminal.

[Insert your own terminal screenshot here]

6. View Data

Purpose: Display the watchlist records in the terminal.

Working / Logic:

1. `check_data()` checks whether the expected CSV data file exists before the script tries to display it.
2. `cat -n` prints file contents with line numbers, which makes individual records easier to refer to.
3. `more` provides page-by-page viewing for longer output; use the terminal's supported key to move through pages.

Commands used: `check_data()`, `cat -n`, `more`.

Output explanation: The terminal shows the CSV header/records that are present in the file, with line numbers. If the data file is missing, the script's data check handles that condition rather than treating the file as normal data.

Screenshot placement: Select menu option 6. Capture the CSV content with line numbers visible.

[Insert your own terminal screenshot here]

7. Backup Data as tar.gz

Purpose: Create a compressed archive and show its size.

Working / Logic:

1. `date` supplies a timestamp used in the backup filename.
2. `tar -czf` archives the `data/` directory as a gzip-compressed `.tar.gz` file in `backups/`.
3. `du -h` reports the archive size in a human-readable unit such as K or M.
4. `cut -f1` extracts the first tab-separated field from the `du` output, which is the size value.
5. `log()` records the operation.

Commands used: `date`, `tar -czf`, `du -h`, `cut -f1`, `log()`.

Output explanation: The terminal displays the archive path and its human-readable size. This feature uses logic similar to Feature 5: both make timestamped compressed backups of `data/`. Based on the described behaviour, Feature 7 additionally reports the file size; verify exact filename and any other differences in the source script.

Screenshot placement: Select menu option 7. Capture both the generated archive path and the displayed file size.

[Insert your own terminal screenshot here]

8. View Activity Log

Purpose: Review recent actions recorded by the program.

Working / Logic:

1. `log()` creates a timestamp using `date` and appends a description of an action to `logs/activity.log`.
2. The `>>` operator appends each new entry to the end of the file instead of replacing earlier entries.
3. Actions may include adding a title, updating a rating, searching recommendations, and creating backups.
4. `tail -n 15` prints the latest 15 lines, allowing the user to review recent activity.

Commands used: `date`, `>>`, `log()`, `tail -n 15`.

Output explanation: The terminal displays up to the last 15 lines currently present in the log file. If fewer than 15 lines exist, it can display fewer.

Illustrative example: Illustrative format only (not an actual log entry): `YYYY-MM-DD HH:MM:SS - Added title: [title]`.

Screenshot placement: Select menu option 8. Capture the latest log lines displayed by your terminal.

[Insert your own terminal screenshot here]

9. Cron Task: Automatic Weekend Recommendations

Purpose: Write up to three randomly selected pending records to `weekend_picks.txt`.

Working / Logic:

1. `grep -i ',pending,'` searches for CSV records containing the pending status field.
2. `sort -R` randomizes the order of the matching lines.
3. `head -n 3` takes up to the first three lines from that randomized list.
4. `> weekend_picks.txt` writes the selected lines to the output file, replacing its previous contents on each run.
5. `log()` records that the cron task ran.

Commands used: `grep -i ',pending,'`, `sort -R`, `head -n 3`, `>`, `log()`, `crontab`.

Output explanation: After the task runs, `weekend_picks.txt` contains up to three pending records. If there are fewer than three pending records, fewer lines may be written. If there are no matches, the output file may be empty.

Schedule: The documented schedule is Saturday at 7:00 PM. Scheduled execution depends on the cron service being active, the user's crontab containing the correct entry, and the job having the correct paths and environment. Running `./main.sh --cron` manually tests the task on demand; it does not prove the scheduled job will run automatically.

Screenshot placement: Run `./main.sh --cron`, then `cat weekend_picks.txt`. For the schedule, run `crontab -l`.

[Insert Screenshot 10: Weekend Recommendations Here] [Insert Screenshot 11: Cron Schedule Here]

4. Terminal Commands to Run the Project

Type these commands in a terminal. They are not all interactive menu options.

Terminal command	When / why to use it
`cd ~/linux_project/watchlist`	Open the project folder. Adjust the path if the project is stored elsewhere.
`ls`	List files and folders in the project root.
`ls lib`	Check the library folder.
`ls data`	Check the data folder.
`chmod +x main.sh`	Set executable permission only if `main.sh` does not already have it.
`./main.sh`	Start the interactive menu.
`./main.sh --help`	Display command-line help.
`./main.sh --backup`	Run the documented command-line backup operation.
`./main.sh --cron`	Run the weekend recommendation task manually.
`cat weekend_picks.txt`	Display the generated weekend recommendations.
`cat -n data/watchlist.csv`	Display CSV data with line numbers.
`tail -n 15 logs/activity.log`	Display the latest 15 activity log lines.
`ls -lh backups/`	List backup files with human-readable sizes.
`crontab -l`	Display the current user's configured cron entries.
`cat README.md`	Read the project instructions.

5. Important Linux Commands

Command	Purpose in this project	Feature where used
read	Accepts input from the user.	Add title; mark watched; genre search
echo	Prints text; with `>>`, appends a record.	Add title
grep	Searches for matching records or titles.	Add; update; recommendations; cron
cut -d',' -f1	Extracts the first comma-separated field (title).	Add title / duplicate check
sed -i	Edits the selected file record in place.	Mark watched and rate
awk -F','	Processes CSV fields for calculations.	Average rating per genre
date	Creates a timestamp for filenames/log entries.	Backups; activity log
tar -czf	Creates a gzip-compressed tar archive.	Both backup features
du -h	Shows a file size in human-readable units.	tar.gz backup

Command	Purpose in this project	Feature where used
cut -f1	Extracts the first tab-separated field.	Backup size display
cat -n	Displays file contents with line numbers.	View data
more	Shows longer output page by page.	View data
tail -n 15	Displays the latest 15 lines.	Activity log
sort -R	Randomizes the order of lines.	Weekend recommendations
head -n 3	Selects up to the first three lines.	Weekend recommendations
>>	Appends output without replacing existing content.	CSV records; activity log
>	Writes output and replaces the target file's contents.	weekend_picks.txt
crontab -l	Lists the current user's scheduled jobs.	Check cron schedule

6. Screenshot Placement Guide

Capture screenshots from your own terminal and replace each marker with the real image.

Screenshot 1 — Main Menu

What to run / show: Run `cd ~/linux_project/watchlist` and then `./main.sh`. Capture the complete menu and its numbered options.

[Insert Screenshot 1: Main Menu Here]

Screenshot 2 — Add a Title

What to run / show: From the running menu, select option 1. Enter a new, non-duplicate title, type, and genre. Capture the successful addition message.

[Insert Screenshot 2: Add a Title Here]

Screenshot 3 — Mark as Watched and Rate

What to run / show: Select option 2. Enter a title that exists in your CSV and a valid rating from 1 to 10. Capture the updated record.

[Insert Screenshot 3: Updated Record Here]

Screenshot 4 — Genre Recommendations

What to run / show: Select option 3. Enter a genre that has pending records in your data. Capture the matching records, or the no-match message if appropriate.

[Insert Screenshot 4: Genre Recommendations Here]

Screenshot 5 — Average Rating

What to run / show: Select option 4. Capture the averages calculated from your actual watched/rated records.

[Insert Screenshot 5: Average Rating Here]

Screenshot 6 — Dated Backup

What to run / show: Select option 5. Capture the actual backup path printed by the program.

[Insert Screenshot 6: Dated Backup Here]

Screenshot 7 — View Data

What to run / show: Select option 6. Capture the CSV records displayed with line numbers/page viewing.

[Insert Screenshot 7: View Data Here]

Screenshot 8 — tar.gz Backup

What to run / show: Select option 7. Capture the actual backup path and human-readable size.

[Insert Screenshot 8: tar.gz Backup Here]

Screenshot 9 — Activity Log

What to run / show: Select option 8. Capture the recent log lines printed by the program.

[Insert Screenshot 9: Activity Log Here]

Screenshot 10 — Weekend Recommendations

What to run / show: Run `./main.sh --cron`, then run `cat weekend_picks.txt`. Capture the command/result and the file contents.

[Insert Screenshot 10: Weekend Recommendations Here]

Screenshot 11 — Cron Schedule

What to run / show: Run `crontab -l`. Capture the configured entry showing the Saturday 7:00 PM schedule, if it is configured.

[Insert Screenshot 11: Cron Schedule Here]

Screenshot 12 — Help

What to run / show: Run `./main.sh --help`. Capture the help information displayed by your script.

[Insert Screenshot 12: Help Output Here]

7. Testing and Validation

These are suggested tests, not claims that the tests have already passed. Record the actual result from your own run.

• Add a new title and check whether it appears in `data/watchlist.csv`.

• Try an empty required input and then a duplicate title; observe whether the program rejects each.

• Update an existing title with a valid rating from 1 to 10 and inspect the record.

• Try an invalid rating and check that it is rejected.

• Search for a genre with pending records, then try a genre with no pending matches.

• Calculate average ratings and compare one genre manually using its watched, rated records.

• Create both backup types and check the resulting files in `backups/`.

• View recent activity and check whether the actions appear in `logs/activity.log`.

• Run `./main.sh --cron` manually and inspect `weekend_picks.txt`.

• Run `crontab -l` and verify the schedule. A listed entry alone does not prove the cron service successfully executed it.

8. Conclusion

Watchlist Manager demonstrates Bash Shell Scripting and practical Linux command-line work. It combines user input and validation with CSV searching and updating, genre-based calculations, compressed backups, activity logging, and task scheduling. Running the suggested tests and adding genuine terminal screenshots will provide evidence of how the project behaves.

Source and verification note

Source-code note: the uploaded material available for this report was an existing five-page DOCX report; the Bash source files were not attached in this upload. Therefore, detailed explanations below follow that report and the feature-specific implementation details supplied in the request. Exact error wording, CSV column order, archive naming, and code-level edge cases should be checked against `main.sh` and `lib/functions.sh` before final submission.